



What is Endianness? Big-Endian & Little-Endian

Last Updated : 23 May, 2024

Computers operate using binary code, a language made up of **0s** and **1s**. This binary code forms the foundation of all computer operations, enabling everything from rendering videos to processing complex algorithms. A single bit is a **0** or a **1**, and eight bits make up a byte. While some data, such as certain English characters, can be represented by a single byte, other data types require multiple bytes. The concept of **endianness** is crucial in understanding how these bytes are read and interpreted by computers.

What is Endianness?

Endianness refers to the order in which bytes are arranged in memory. Different languages read their text in different orders. For example, English reads from left to right, while Arabic reads from right to left. **Endianness** works similarly for computers. If one computer reads bytes from left to right and another reads them from right to left, issues arise when these computers need to communicate.

Endianness ensures that bytes in computer memory are read in a specific order. Each computer system is internally consistent with its own data, but the advent of the internet has led to more data sharing than ever before, and not all systems read data in the same order.

Endianness comes in two primary forms: Big-endian (BE) and Little-endian (LE).

- **Big-endian (BE):** Stores the most significant byte (the "big end") first. This means that the first byte (at the lowest memory address) is the largest, which makes the most sense to people who read left to right.
- **Little-endian (LE):** Stores the least significant byte (the "little end") first. This means that the first byte (at the lowest memory address) is the smallest, which makes the most sense to people who read right to left.

What is Big-endian?

In a big-endian system, the **most significant byte (MSB)** is stored at the lowest memory address. This means the "big end" (the most significant part of the data) comes first. For instance, a 32-bit integer 0x12345678 would be stored in memory as follows in a big-endian system:

Address:	00	01	02	03
Data:	12	34	56	78



Here, **0x12** is the most significant byte, placed at the lowest address (**00**), followed by **0x34**, **0x56**, and **0x78** at the highest address (**03**).

What is Little-endian?

A little-endian system stores the **least significant byte (LSB)** at the lowest memory address. The "little end" (the least significant part of the data) comes first. For the same 32-bit integer 0x12345678, a

little-endian system would store it as:

```
Address:  00  01  02  03
Data:      78  56  34  12
```



Here, **0x78** is the least significant byte, placed at the lowest address (**00**), followed by **0x56**, **0x34**, and **0x12** at the highest address (**03**).

Significance of Most Significant Byte (MSbyte) in Little and Big Endian:

Understanding the concept of the **Most Significant Byte (MSbyte)** helps clarify endianness further. Let's use a decimal number to illustrate.

Consider the decimal number 2,984. Changing the digit 4 to 5 increases the number by 1, while changing the digit 2 to 3 increases the number by 1,000. This concept applies to bytes and bits as well.

Tutorial Interview Questions Quizzes Must Do Advanced DSA System Design Aptitude Puzzles Interview Corner DSA Pyt



- **Least Significant Byte (LSbyte):** The byte that holds the lowest position value.

In big-endian format, the MSbyte is stored first. In little-endian format, the MSbyte is stored last.

When Might Endianness Be an Issue?

Endianness must be considered in various computing scenarios, particularly when systems with different byte orders need to communicate or share data.

1. **Unicode Characters:** Unicode, the character set used universally across devices, uses a special character byte sequence called the **Byte Order Mark (BOM)**. The **BOM** informs the system that the incoming stream is Unicode, specifies which Unicode character encoding is used, and indicates the endian order of the incoming stream.
2. **Programming Languages:** Some programming languages require specifying the byte order sequence. For instance, in **Swift**, used for **iOS** development, you can define whether data is stored in **big-endian** or **little-endian format**.
3. **Network Protocols:** Different protocols have emerged historically, leading to the need for interaction. **Big-endian** is the dominant order in network protocols and is referred to as network order. Conversely, most PCs use **little-endian** format. Ensuring interoperability between these formats is critical in network communication.
4. **Processor Design:** Processors can be designed to be either **little-endian**, **big-endian**, or **bi-endian** (capable of handling both). Consumer choice and the resulting market trends have influenced what is considered "normal" in computer systems today.

Why is Endianness an Issue?

Endianness becomes an issue primarily due to the interaction between different systems and protocols. Historical protocol development led to varying byte order conventions, necessitating data conversion for compatibility. In higher-level languages and abstracted environments, endianness is often managed behind the scenes, reducing the need for developer concern. However, understanding endianness remains crucial for low-level programming, network protocol design, and data interoperability.

Conclusion

Endianness is how bytes are ordered in computer data. **Big-endian** and **little-endian** are two ways to arrange bytes, each with advantages. Understanding endianness is very important for developers dealing with low-level data, networking, and system interoperability. While **little-endian** is common, both formats remain important as technology evolves. Strategies for managing data across endian conventions will continue developing to ensure compatibility and performance.

Comment

K

kartik

154

Article Tags: [Bit Magic](#) [DSA](#) [Little Endian](#) [Big Endian](#)

+1 More

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Courses



- Corporate & Communications Address:**
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)
- Registered Address:**
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company	Explore	Tutorials	Courses	Videos	Preparation Corner
About Us	POTD	Programming Languages	ML and Data Science	DSA	Interview
Legal	Job-A-Thon	DSA	DSA and Placements	Python	Corner
Privacy	Blogs	Web Technology	Web Development	Java	Aptitude
Policy	Nation	AI, ML & Data Science	Programming Languages	C++	Puzzles
Contact Us	Skill Up	DevOps	DevOps & Cloud	Web Development	GfG 160
Advertise with us		CS Core Subjects	GATE	Data Science	System Design
GFG Corporate Solution		Interview Preparation	Trending Technologies	CS Subjects	
Training Program		Software and Tools			

